

UNIVERSIDADE POSITIVO

Caio Voitena Roupá
João Alexandre Vilaruel dos Santos

Gerenciamento Esportivo API

Curitiba 12/06/2025
Sala: 404

DOCUMENTAÇÃO TÉCNICA DO PROJETO – API GERENCIAMENTO ESPORTIVO

Título do Projeto: API Gerenciamento Esportivo

Grupo: João Alexandre Vilaruel dos Santos, Caio Voitena Roura e

1. Objetivo do Projeto

O objetivo deste projeto é desenvolver uma API para gerenciamento esportivo, que permita o controle eficiente de informações relacionadas a esportes, times e jogadores. A aplicação possibilita operações como consulta, cadastro, atualização e exclusão das entidades mencionadas, proporcionando uma base estruturada para uso em sistemas esportivos, ligas ou clubes que precisam gerenciar dados desses elementos de forma organizada e acessível.

2. Estrutura da Solução

2.1 Modelagem de Dados

A modelagem do sistema é centrada em três entidades principais: Esporte, Time e Jogador.

- Esporte: Representa o tipo de esporte (ex: futebol, basquete), contendo atributos como Id e Nome.
- Time: Representa os times que participam dos esportes, contendo Id, Nome, Ano de Fundação, Quantidade de Títulos, Lista de Títulos e EsporteId (para referenciar o esporte).
- Jogador: Representa os atletas, com atributos como Id, Nome, Idade, Número da Camisa, Salário, TimeId e EsporteId (para relacionamento com time e esporte).

Essas entidades foram modeladas em classes na camada de domínio, que servem de base para a persistência dos dados utilizando arquivos texto, mantendo simplicidade e facilidade de manutenção.

2.2 Integração da API com a Solução

A API funciona como interface para manipulação dos dados, expondo endpoints REST para cada entidade. Cada rota permite operações de CRUD (Create, Read, Update, Delete), facilitando a integração com clientes externos como front-ends, sistemas terceiros ou ferramentas de testes.

3. Endpoints da API

Método	Rota	Descrição
--------	------	-----------

GET	/api/esportes	Retorna todos os esportes
GET	/api/esportes/{id}	Retorna um esporte por ID
POST	/api/esportes	Adiciona um novo esporte
PUT	/api/esportes/{id}	Atualiza um esporte existente
DELETE	/api/esportes/{id}	Remove um esporte por ID

Endpoints para Times:

Método	Rota	Descrição
GET	/api/times	Retorna todos os times
GET	/api/times/{id}	Retorna um time por ID
POST	/api/times	Adiciona um novo time
PUT	/api/times/{id}	Atualiza um time existente
DELETE	/api/times/{id}	Remove um time por ID

Endpoints para Jogadores:

Método	Rota	Descrição
GET	/api/jogadores	Retorna todos os jogadores
GET	/api/jogadores/{id}	Retorna um jogador por ID
POST	/api/jogadores	Adiciona um novo jogador
PUT	/api/jogadores/{id}	Atualiza um jogador existente
DELETE	/api/jogadores/{id}	Remove um jogador por ID

4. Organização do Código

O projeto foi organizado de forma modular para facilitar manutenção e entendimento:

- Models/: Define as entidades Esporte, Time e Jogador com seus respectivos atributos.
- Repositories/Interfaces/: Define interfaces para acesso e manipulação dos dados (ex: IEsporteRepository, ITimeRepository, IJogadorRepository).
- Repositories/Implementations/: Implementações concretas das interfaces que realizam a persistência dos dados em arquivos texto.
- Services/: Contém a lógica de negócio para as operações das entidades, utilizando os repositórios.
- Endpoints/: Controladores que expõem as rotas REST da API, recebendo requisições e respondendo com os dados ou mensagens adequadas.
- Program.cs: Arquivo principal de inicialização que configura serviços, repositórios, middlewares (como Swagger) e rotas da aplicação.
- Frontend/: Pasta public contendo index.html, style.css e script.js que integram a funcionalidade visual da API

5. Justificativa Técnica

A escolha de utilizar arquivos texto como persistência visa simplicidade e facilidade de distribuição, atendendo ao escopo inicial do projeto para fins acadêmicos e de demonstração. A arquitetura em camadas, com separação clara entre modelos, repositórios, serviços e endpoints, promove manutenibilidade, extensibilidade e facilita testes.

O uso do Swagger para documentação da API auxilia na compreensão e validação das rotas disponíveis, tornando o desenvolvimento mais ágil e organizado.

Essa estrutura permite futuras melhorias, como migração para banco de dados relacional, autenticação de usuários, ou inclusão de novas entidades e funcionalidades, mantendo o padrão arquitetural adotado.

6. Considerações Finais

Este projeto foi importante para aplicarmos conceitos de desenvolvimento de APIs e organização de código. Optamos por arquivos texto para a persistência por simplicidade e agilidade, atendendo ao escopo acadêmico.

Dividimos o código em camadas, o que facilitou a manutenção e a compreensão do sistema. Usamos o Swagger para documentar e testar a API, garantindo que as rotas funcionassem corretamente.

O grupo realizou testes manuais para verificar o funcionamento das funcionalidades, assegurando a qualidade do sistema.

Embora seja um projeto acadêmico, a estrutura permite futuras melhorias, como uso de banco de dados, autenticação e novas funcionalidades.

Por fim, o trabalho em equipe, o planejamento e os testes foram essenciais para o desenvolvimento bem-sucedido do projeto e para nosso aprendizado.

7. Integração Front-end – Interface Web

Para facilitar o uso da API e demonstrar sua aplicação prática, foi desenvolvido um front-end web simples, utilizando HTML, CSS e JavaScript puro, que consome os endpoints REST da API.

8. 8.1 Estrutura do Front-end

O front-end é composto por três seções principais, cada uma responsável pelo gerenciamento de uma entidade: Esportes, Times e Jogadores. Cada seção possui um formulário para cadastro e uma lista para exibição, exclusão e edição dos registros.

9. 8.2 Estilo Visual (CSS)

O CSS foi desenvolvido para proporcionar uma interface moderna, responsiva e agradável, utilizando cards, botões destacados e listas organizadas.

10. 8.3 Lógica JavaScript

O JavaScript realiza as operações de CRUD consumindo a API via fetch, atualizando as listas e selects dinamicamente.

11. 8.4 Integração e Funcionamento

O front-end consome os endpoints REST da API para listar, adicionar, editar e excluir esportes, times e jogadores.

Os selects são preenchidos dinamicamente para garantir o relacionamento correto entre entidades.

O botão "Adicionar" envia os dados do formulário para a API, o botão "Excluir" remove o registro correspondente e o botão "Editar" edita os dados, altera e salva.

O layout é responsivo e pode ser utilizado em diferentes dispositivos.

12. 8.5 Observações Técnicas

CORS: Certifique-se de que o CORS está habilitado na API para permitir requisições do front-end.

Endereço da API: O endereço da constante API no JS deve ser ajustado conforme o endereço/porta em que a API está rodando.

Validação: O front-end faz validação básica dos campos obrigatórios, mas recomenda-se validar também no back-end.